# AFlow: Automating Agentic Workflow Generation

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, et al. · ICLR 2025

DOI: 10.48550/arXiv.2410.10762

Summary

Represent workflows as code, search the code space automatically via Monte Carlo Tree Search, and use execution scores as the feedback signal. It outperforms manual and prior automated methods across six benchmarks, and lets weaker models outperform stronger ones at much lower cost.

2026.10.29

# Contents


01 Background & Pain Points P03

02 Prior Approaches & the Breakthrough P04

03 **Method:** Formalization & the Search Loop P05-09

04 Experiments P10-13

05 Limitations P14-15

PART 1 · Background & Pain Points

# A smart model still needs a human to draw the flowchart

**Agentic workflow** — a pre-orchestrated pipeline of LLM calls: Generate → Verify → Revise → Vote. CoT, Self-Refine and Multi-Agent Debate are all *fixed* pipelines: a human decided the structure before the model ever ran.

### the human draws it

Plan → Solve → Check → Vote

1 · **Not scalable** — every new domain means a full redesign; human cost grows with each task.

2 · **Not transferable** — a pipeline tuned on math fails on code or QA.

3 · **Expert intuition** — where to add verification or voting is trial-and-error.

Intuition can backfire: Self-Refine **70.7** < plain IO **72.8** (Table 1, six-benchmark avg).

### the machine searches it

Search space × Reward signal → Workflow found by search

· One search per task and budget — no human in the loop.

· The output is a **structure**, so it can be reused and transferred.

· The process is reproducible: same space, same signal, same result.

This is exactly how search beat human design in architecture and in prompts — AFlow moves it to the workflow level.

So the paper's goal is : *given task T and evaluation G, produce a good workflow automatically.*

# PART 2 · Prior Approaches & the Breakthrough

TIER 1 **Prompt optimization**

DSPy · TextGrad · OPRO

Structure is fixed; only node prompts are tuned. Cheap and easy — with an obvious performance ceiling.

TIER 2 **Fixed-architecture tuning**

Archon

Tunes parameters inside a predefined architecture (ensembles, voting) ,a new task still needs a human to pick the architecture.

TIER 3 **Workflow structure — ADAS**

this paper's arena

Searches prompts + edges as code — the **right space**, but the wrong search and the wrong memory.

## The ADAS trap · one long list

workflow w1 · score 0.49

workflow w2 · score 0.52

workflow w3 · score 0.43

... all past complete workflows ...

workflow wn · score 0.53

→

**Optimizer prompt**

everything, in generation order, scores attached — **context flooded, signal drowned.**

**Linear heuristic search:** every candidate lives in one flat population; there is no notion of "this branch is promising, go deeper".

## paper's own evidence

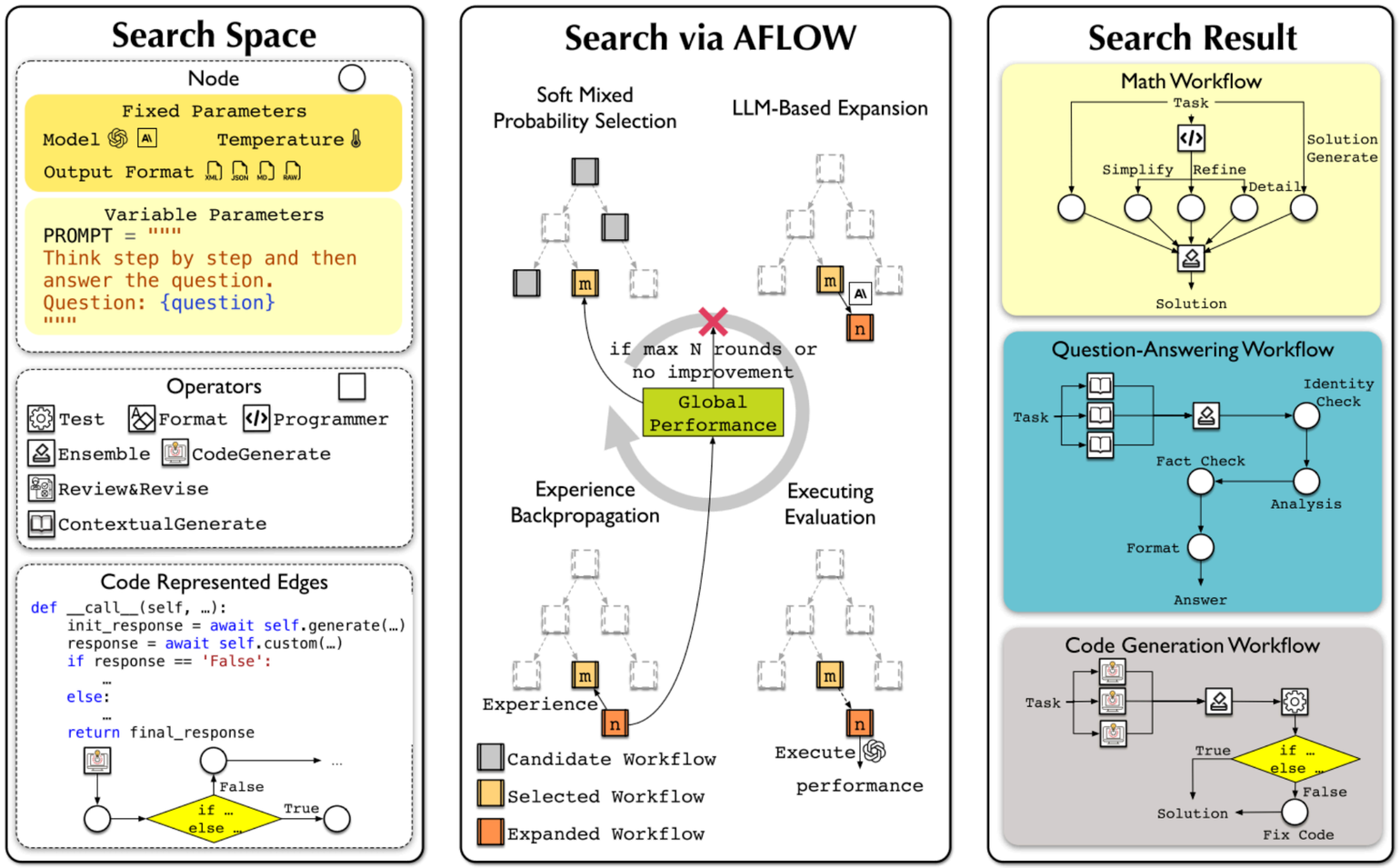
On HotpotQA, ADAS's best workflow **adds roles and a retrieval step that inflate multi-hop questions** — when the right move was stepwise reduction to single hops.

Result in Table 1: ADAS averages 67.2 — **below plain IO at 72.8.** Automation that forgets structure can end up worse than no automation.

The optimizer could not tell which past edit helped where, because experience carried no structure — only order and a score.

So the bottleneck is not **“ can a LLM write a workflow ”** — it is **“ How experience is organized during search ”** .

# PART 3 · Method: Formalization & the Search Loop




Paper Figure 3: the overall AFlow framework — search space (left), the MCTS-style loop (middle), workflows found for three tasks (right).

## ① Representation · workflow = code

Nodes, operators and edges written in Python — branches and loops become native, and the LLM can edit text far more reliably than it mutates a graph.

## ② Search · a variant of MCTS

A tree of candidate workflows, not a flat population; expansion is done by an LLM acting as the mutation operator — like AlphaGo, it tries and remembers.

## ③ Signal · execution feedback

Run the candidate on validation data, score it, and backpropagate the diff — "what was changed, did it help" — to the parent node.

# PART 3 · Formalization

NODE $n = N(M, \tau, P, F)$

$M$ model · $\tau$ temperature · $P$ prompt · $F$ output format. The sharp choice: **the output format is a variable** — enforced formats carry a measured performance cost (Tam et al., 2024), so a format deserves to be optimized, not assumed.

SPACE $S = \{(N, E) \mid E \in E\}$

A workflow is a set of nodes $N$ wired by edges $E$; the space contains **every wiring**. This is what "search the structure" means formally.

OBJECTIVE $W^* = \operatorname{argmax} G(W, T), \quad W = A(S, G, T)$

$G$ is the evaluation function on task $T$; $A$ is the search algorithm. **Any method with a computable score can be plugged in here** — this formulation is the paper's most reusable artifact.

## One formulation to rule them all

prior methods are subspaces of S

- **DSPy / TextGrad:** fix E, M, τ, F — search only P
- **Archon:** fix the structure — tune inside it
- **ADAS:** search P + E — but with linear search
- **AFlow:** search E + P — with MCTS over code

Read the three boxes as one sentence: *a workflow is a wired set of typed nodes; the space is all wirings; the goal is the wiring with the best score.*

# PART 3 · Representation

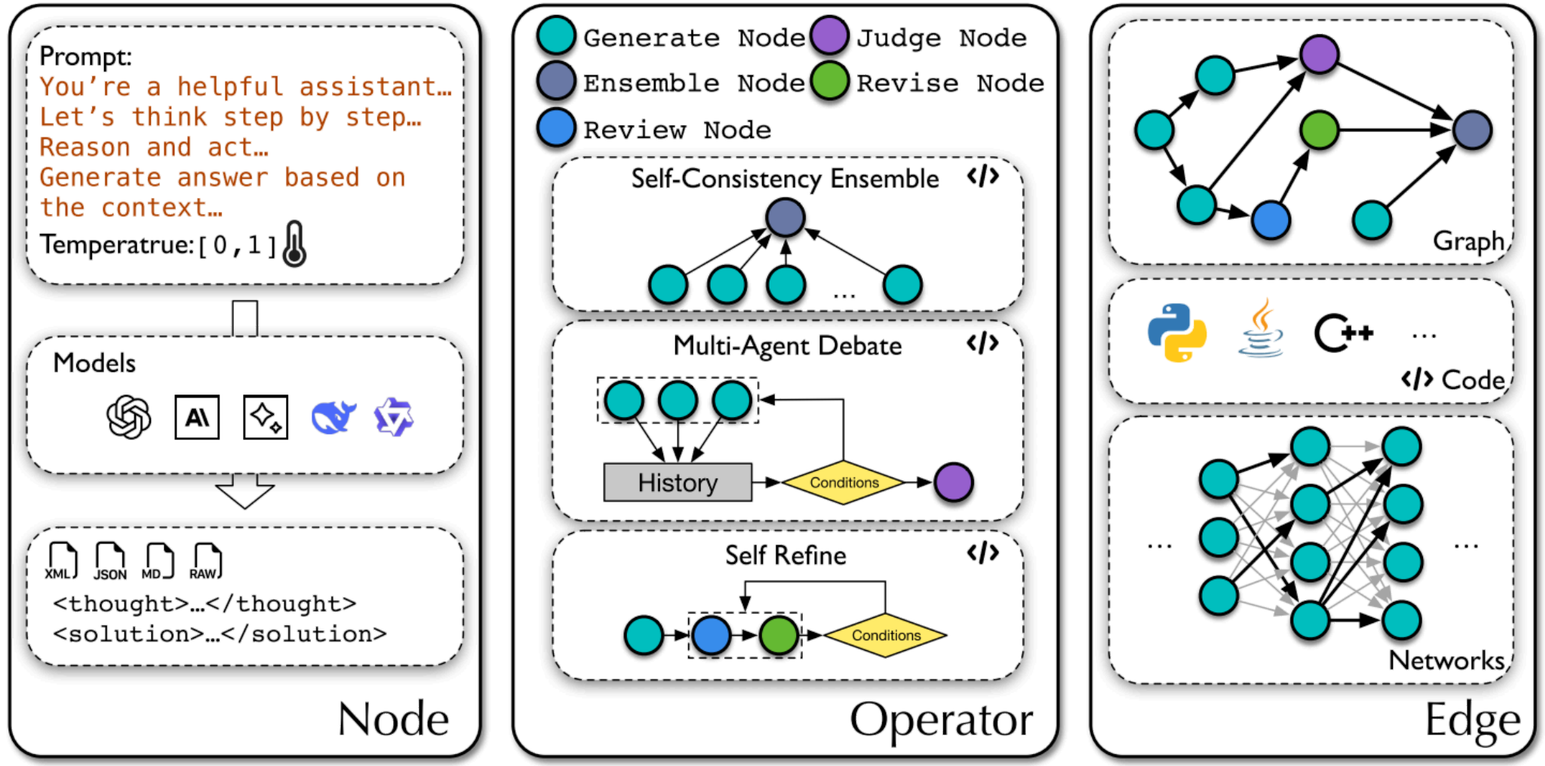



Figure 2: examples of nodes, operators and edges — the vocabulary of the search space

### Node · even the output format is a variable

Model / prompt / temperature / format. A variable with a measured cost belongs in the search space, not in the assumptions.

### Operator · manual tricks, modularized

Ensemble, Review, Revise… = pre-wired node+edge units. Human priors that speed the search — and quietly bound what it can reach (Slide 11 tests this).

### Edge · code vs graph vs text

Graphs need bolt-ons (Petri nets) for branches; text cannot hold dependencies at all. Code natively expresses sequence, branch and loop — and is what the LLM edits best.

**The argument in one line:** code can express every graph plus everything graphs cannot — expressiveness is claimed as a *strict superset*, which is the strongest form such an argument can take.

# PART 3 · Algorithm

### ① Pick

Soft-mixed probability over the top-k scored workflows + the blank template — keeps exploration permanently alive.

→

### ② Edit

Claude-3.5-Sonnet mutates **one thing**: a prompt or the code edges — small, reviewable diffs, not rewrites.

→

### ③ Taste

Execute 5× on the validation set; record mean ± std. Costly per round — bought accuracy to save rounds.

→

### ④ Record

Score + diff + success/fail go back to the *parent*; the score enters the global record for future selection.

EQ. 3 **Soft-mixed probability selection — the "pick" step, written out**

$$P_{\text{mixed}}(i) = \lambda \cdot (1/n) + (1-\lambda) \cdot \exp(\alpha (s_i - s_{\max})) \,/\, \sum_j \exp(\alpha (s_j - s_{\max})) \quad \alpha = 0.4, \lambda = 0.2$$

**λ·1/n — the 20% floor:** pure uniform exploration, so the blank template and long shots never die out.

**softmax term — the 80%:** exploit high scorers, with $\alpha$ controlling how sharp the preference is.

**Subtracting s-max:** numerically stabilizes exp — a one-token detail that shows this was built, not just written.

**Terminal condition:** early stop when the top-k average shows no improvement for n consecutive rounds, or after N rounds (Appendix A.6) — the paper does price wasted rounds here, just not anywhere else.

Design intuition: MCTS supplies *where to look next*, the LLM supplies *what to try there*, the execution score supplies *whether it worked*.

# PART 4 · Does It Work?

| Method | Benchmarks | | | | | | Avg. |
|---|---|---|---|---|---|---|---|
| | HotpotQA | DROP | HumanEval | MBPP | GSM8K | MATH | |
| IO (GPT-4o-mini) | 68.1 | 68.3 | 87.0 | 71.8 | 92.7 | 48.6 | 72.8 |
| CoT (Wei et al., 2022) | 67.9 | 78.5 | 88.6 | 71.8 | 92.4 | 48.8 | 74.7 |
| CoT SC (5-shot) (Wang et al., 2022) | 68.9 | 78.8 | 91.6 | 73.6 | 92.7 | 50.4 | 76.0 |
| MedPrompt (Nori et al., 2023) | 68.3 | 78.0 | 91.6 | 73.6 | 90.0 | 50.0 | 75.3 |
| MultiPersona (Wang et al., 2024a) | 69.2 | 74.4 | 89.3 | 73.6 | 92.8 | 50.8 | 75.1 |
| Self Refine (Madaan et al., 2023) | 60.8 | 70.2 | 87.8 | 69.8 | 89.6 | 46.1 | 70.7 |
| ADAS (Hu et al., 2024) | 64.5 | 76.6 | 82.4 | 53.4 | 90.8 | 35.4 | 67.2 |
| Ours | **73.5** | **80.6** | **94.7** | **83.4** | **93.5** | **56.2** | **80.3** |

Table 1: Comparison of performance between manually designed methods and workflow generated by automated workflow optimization methods in QA, code, and Math scenarios. All methods are executed with GPT-4o-mini on divided test set, and we tested it three times and reported it on the Average.

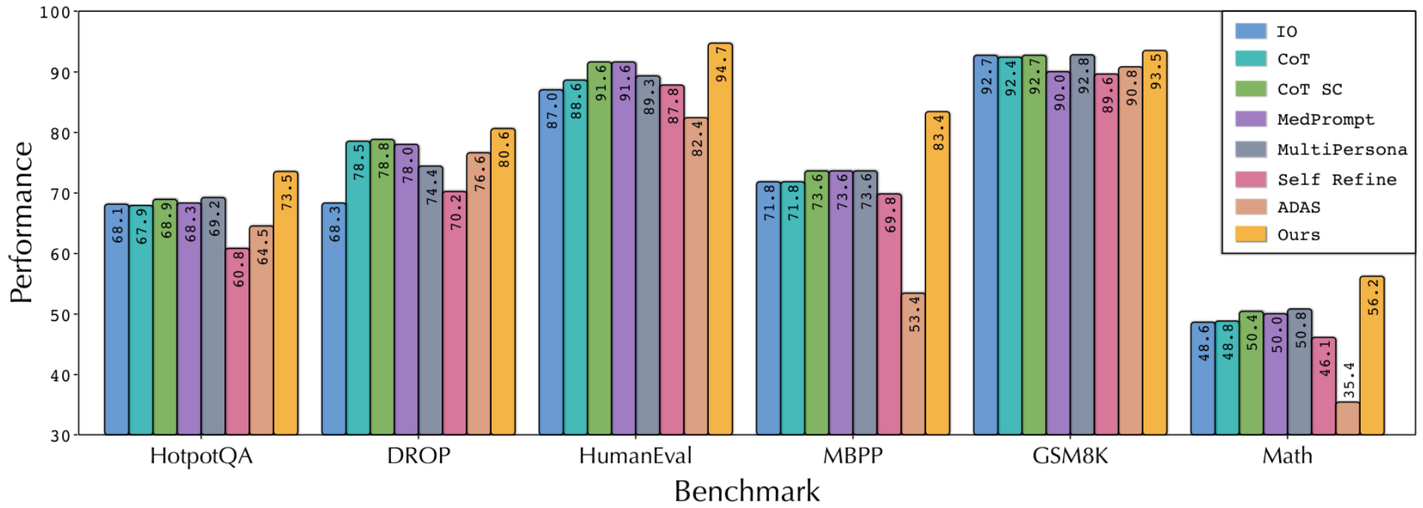



Figure 1: performance across six benchmarks (GPT-4o-mini executor) — "Ours" (orange) leads every group

## +5.7%

over the best human-designed method — 80.3 six-benchmark average, vs ~76 for the strongest manual pipeline

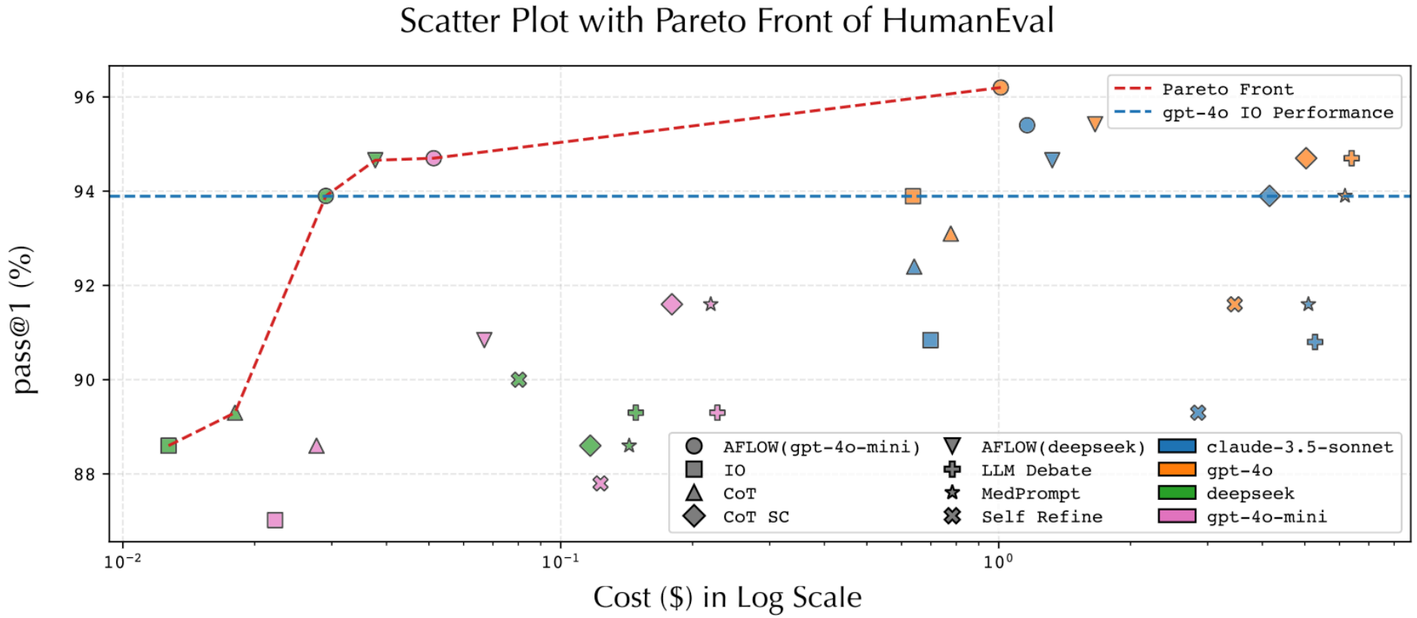
## +19.5%

over ADAS, the prior automated method — 80.3 vs 67.2. Flag for Slide 13: ADAS sits below plain IO at 72.8.

## Hardest tasks

widest gaps where structure matters most: MBPP 83.4 vs 73.8 next-best; MATH 56.2 vs 50.8 — search pays off exactly where hand design is hardest

# PART 4 · Cost & Transfer




Paper Figure 4: cost – performance on HumanEval; red dashed = Pareto front, blue dashed = GPT-4o IO reference

| Model | Methods | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | IO | CoT | CoT SC | MP | MPD | SR | Ours | Ours* |
| GPT-4o-mini | 87.0 | 88.6 | 91.6 | 91.6 | 89.3 | 87.8 | 94.7 | 90.8 |
| DeepSeek-V2.5 | 88.6 | 89.3 | 88.6 | 88.6 | 89.3 | 90.0 | 93.9 | 94.7 |
| GPT-4o | 93.9 | 93.1 | 94.7 | 93.9 | 94.7 | 91.6 | **96.2** | 95.4 |
| Claude-3.5-sonnet | 90.8 | 92.4 | 93.9 | 91.6 | 90.8 | 89.3 | 95.4 | 94.7 |

A searched workflow beats every LLM: GPT-4o-mini ( " Ours " ) and DeepSeek-V2.5 ( " Ours* " ).

And the structure transfers — Table 2 (HumanEval, 3-run avg)

### Where 4.55% comes from (Appendix D)

GPT-4o direct: 93.89 / $0.6371

DeepSeek-V2.5 + searched workflow: 93.90 / $0.0291

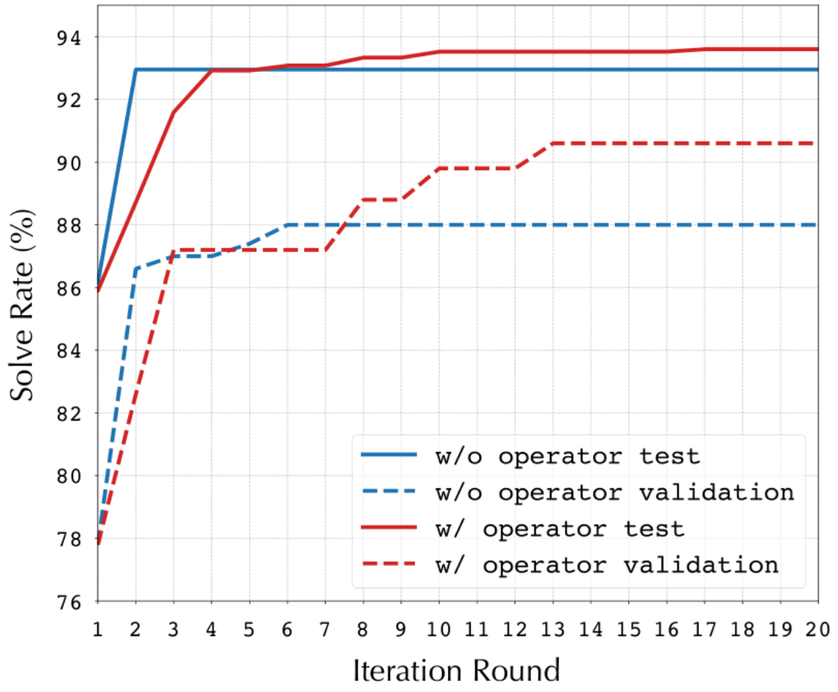
$$0.0291 \div 0.6371 \approx 4.55\%$$

A 20×-cheaper executor with a searched workflow **matches** the strongest model's plain call — "what to run" matters as much as "which model runs it".

"different models require different workflows"

# PART 4 · Ablation

The Test and Validation Curve of GSM8K




(A)

The Code Represented Workflow of GSM8K

```python
async def __call__(self, problem: str):
"""
Implementation of the graph
"""
    solutions = []
    for _ in range(5): # Generate 5 solutions
        solution = await self.custom(
            input=problem, instruction=MATH_SOLVE_PROMPT
        )
        solutions.append(solution['response'])

    final_solution = await self.sc_ensemble(
        solutions=solutions, problem=problem
    )
    # Add a verification step using Programmer operator
    verification = await self.programmer(
        problem=problem,
        analysis=final_solution['response']
    )

    if verification['output']:
        return verification['output'], self.total_cost
    else:
        return final_solution['response'], self.total_cost
```

(B)

Figure 5: validation curves with vs without operators (left); the best workflow AFlow found for GSM8K without any operators (right)

## 93.1%

final score with **zero operators** on GSM8K — still above every manual method in Table 1. The search engine, not the human bricks, is the source of the gains.

### Operators = accelerant, not fuel

With operators the curves rise faster (left panel); without them AFlow still invents its own ensemble and verification (right panel shows 5 × Custom generate-check nodes).

### But read the fine print

The ablation covers one dataset (GSM8K — chosen because its size makes each point cheapest to buy) and reports curves without tables. Broaden it before believing it everywhere.

This is the paper's most honest self-assessment: *human priors accelerate but do not create the gains*

# PART 4 · Inside one search

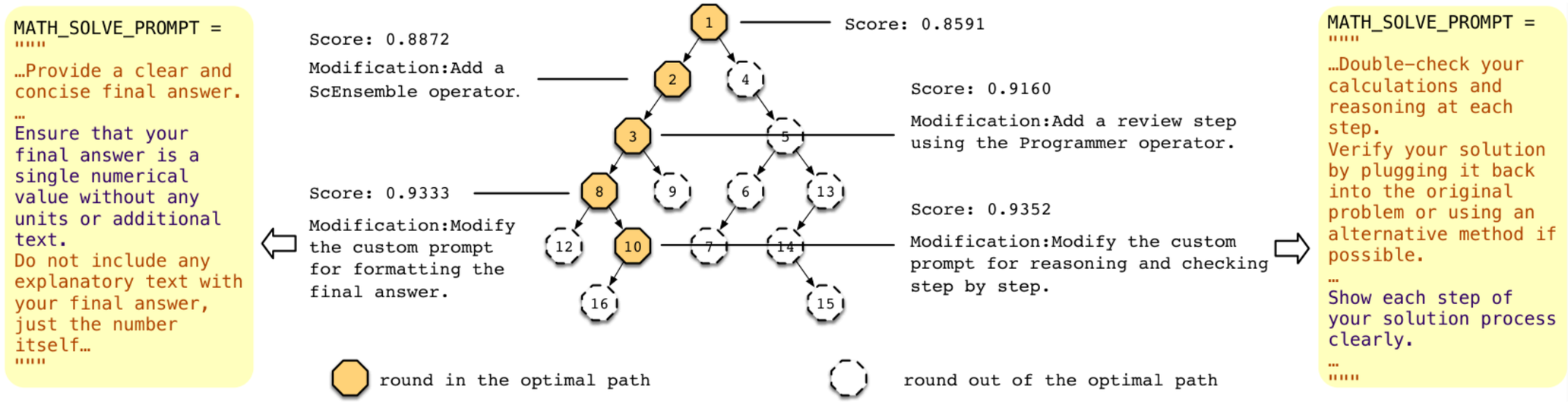



Figure 6: the path from round 1 to the best GSM8K workflow; solid nodes = optimal path, dashed = pruned; purple text = the actual prompt edits

blank · 0.8591 → + ScEnsemble · 0.8872 → + Programmer verify · 0.9160 → format prompt · 0.9333 → **reason & check prompt · 0.9352**

### Read the score deltas, not the nodes

Structural moves buy the big points (+2.8, then +2.9); prompt edits refine at the end (+1.7, then +0.2). The search discovered the same build order a competent human would follow — **structure first, polish later**.

### One figure doing two jobs

The side-by-side prompts (purple = what changed) double as an ablation of the winner: you can see exactly which two sentences carry the last points. In a talk, this is the slide to slow down on — it answers "what did the search actually produce?" better than any table.

# PART 5 · Limitations

## 1 · Search cost is never priced

5 validation runs × 20 rounds + optimizer calls are absent from Figure 4; the economics rest on an unstated "reuse one workflow many times".

**Fix:** report $/point and a search-investment curve.

## 2 · The space is narrower than promised

M, τ, F are locked (Sec. 3.2); all 7 operators are hand-defined. "Unify prior methods" really means searching a hand-built subspace.

**Fix:** put model choice and operator synthesis into the space.

## 3 · The one automated baseline looks under-tuned

ADAS averages 67.2, below plain IO at 72.8, and collapses on the hard tasks: 53.4 on MBPP, 35.4 on MATH. With a single automated baseline, the +19.5% headline is fragile.

**Fix:** re-tune ADAS; add a second automated baseline.

## 4 · One optimizer, never varied

Every workflow was searched by Claude-3.5-Sonnet; Table 2 varies only the executor, so optimizer-dependence is untested.

**Fix:** repeat the search with 2 – 3 optimizers under equal budget.